Group Assignment

Topic: Plan, Build, Ship and Operate a Product as a Scrum Team on Azure

Deadline: 2026-11-24 23:59 (code and documentation). Sprint Reviews in class on **25 and 26 November**.

Weight: 15% of the course grade.

Objective

In teams of 3 to 7, build a **minimum viable product** (MVP) and run it the way a small DevOps team would: planned in short Scrum sprints, delivered through an automated pipeline, deployed on Azure, and observable in production.

Minimum means minimum, **viable** means working. A small product that works, is tested, deploys itself and can be monitored scores higher than an ambitious one that does not run. Keep `main` deployable at all times; add features only on top of a working MVP.

Build something new together. Do not extend anyone's individual assignment. The stack is the same as in Assignment 2 (containers, Azure SQL Database serverless, GitHub Actions, GHCR, Azure Container Apps, Static Web Apps), so nothing new has to be learned; the product is new.

1. Timeline

The project runs in parallel with Assignment 2, so the sprints are short and the scope must be small.

Sprint	Dates	Goal	In class
Sprint 0	4–10 Nov	Team, roles, product idea, initial Product Backlog, repository, Definition of Done, local skeleton (`docker compose up` returns a page or `/health`)	S23: groups formed
Sprint 1	11–18 Nov	Skeleton deployed to Azure by the pipeline, then the first core feature end to end	S26: Sprint 1 Planning
Sprint 2	19–24 Nov	MVP complete, monitoring, documentation	S30: Sprint 1 Review + Retrospective, Sprint 2 Planning
Review	25–26 Nov	Live demo and Sprint Review	S31–S32

Sprint 0 is preparation. The most important milestone is a "Hello World" deployed to Azure by the pipeline in the first days of Sprint 1: everything after it is iteration.

2. Team and Scrum

- **Roles:** one Product Owner and one Scrum Master, rotating at each sprint; everyone is a Developer. Record who held each role in each sprint.
- **Backlog:** GitHub Projects in the team repository. Items are user stories with acceptance criteria; the sprint backlog is the items moved into the sprint at Planning.
- **Events:** Planning, Review and Retrospective for each sprint, with short written outcomes (`docs/scrum/sprint-N.md`): Sprint Goal, items selected, items done and not done, retrospective agreements. Daily Scrums are not documented.
- **Definition of Done** in `docs/scrum/definition-of-done.md`, applied to every item. It must at least require: merged to `main` through a reviewed pull request, CI green, deployed.
- **Pull requests:** every change reaches `main` through a pull request reviewed and approved by another team member. No direct pushes to `main` (enable branch protection).

During the Sprint Reviews you act as both team and stakeholders: you present to the class and give feedback to other teams.

3. Product Requirements

3a. Hard requirements:

- A **web application** with a static UI on Azure Static Web Apps and at least **one persisted entity with full CRUD** and **one feature beyond CRUD** (a search, a recommendation, a calculation, an LLM call, an integration with a public API…).
- **Backend:** Python (Flask or FastAPI) or Node.js (Express). **Frontend:** static files (plain HTML/JS, or React/Vue built to static files).
- **Architecture: exactly two services**, each in its own container with its own database, talking over HTTP; the UI is a static site that calls them from the browser (CORS restricted to its origin), as in Assignment 2 §1a. Inside each service the structure (layered, hexagonal, modular) is your choice. Justify the split and the structure (§7).
- **Containers:** a Dockerfile per container (no `COPY . .`, no secrets in the image, pinned dependencies, non-root user) and a `docker-compose.yml` that brings the system up from a clean clone.
- **Migrations:** the schema is defined by versioned migration files applied on startup, as in Assignment 2 §4.
- **`GET /health`** on every service and one log line per request to stdout (method, path, status, duration).
- **No secrets in the repository.** The leaked-secrets procedure of Assignment 2 §5 applies.

3b. Free choices, justify each in `docs/decisions.md`: language and framework, architecture, data model, frontend, how the non-CRUD feature works. If the product calls an LLM or another paid API, its key is a GitHub secret and its cost is kept low (cache, rate limit).

3c. Out of scope: the §1c list of Assignment 2 (always-on compute, Kubernetes, message brokers, other databases, a third container), user authentication beyond a simple mechanism (an optional API key or a hard-coded demo user is enough), and payments.

4. Azure

The **Assignment 2 deployment contract (§7) applies in full**, in the same shared subscription: Azure Container Apps on the Consumption plan scaling to zero (maximum `1` replica), **one serverless Azure SQL database per service** (two services, two databases), with auto-pause (SQL Server in compose locally, never the Azure database from CI), images in GitHub Container Registry, **one Static Web App per group** for the UI (`<prefix>-ui`), configuration only through environment variables and secrets, nothing changed manually after the initial setup. The initial setup is scripted (Azure CLI) or written as IaC (Bicep or Terraform); IaC is credited.

Four differences:

- **Resource group:** each group has its own, already created, `rg-sdd-<class>-g<N>` with the group number on Blackboard (e.g. `rg-sdd-3a-g3` for group 3 of BCSAI 3A). Every member has Contributor access to it with their IE account. Create all the group's resources there, at most **2 Container Apps, 2 serverless SQL databases and 1 Static Web App**; do not create resource groups.
- **Credentials:** the pipeline uses your group's own **service principal**, sent by the professor to the group, with Contributor on the group's resource group only. Not your class's service principal from Assignment 2.
- **Naming:** resources start with your group prefix, `sdd-<class>-g<N>`, instead of your username (e.g. `sdd-3a-g3-api`, `sdd-3a-g3-api-db`, `sdd-3a-g3-ui`). The class is part of the prefix because group numbers repeat across classes.
- **Monitoring:** use the Log Analytics workspace of the Container Apps environment (§6). Keep log volume low: INFO level, no request bodies.

No other Azure services, no other cloud and no managed app platforms (Render, Vercel, Railway, Heroku).

5. CI/CD

GitHub Actions, in the team repository:

On every pull request and push (CI): lint, format check, tests with coverage (**fail below 70%** on the core logic of each service), image build. Recommended: a dependency or static security scan (e.g. `pip-audit`, `npm audit`, Bandit, CodeQL).

On merge to `main` (CD): push images tagged with the commit SHA, deploy the services and the UI to Azure only if CI passed, apply migrations, and check `/health` after the deploy, failing the run if it does not return `200` in time.

Credited extras: IaC for the Azure resources; a release or tagging scheme; a zero-downtime strategy using Container Apps revisions (blue-green or a canary with traffic splitting); a smoke or end-to-end test against the deployed URL.

6. Monitoring and Reliability

- **Request logs** visible in Azure through Log Analytics queries.
- **A dashboard** with at least request count, error rate and response time: saved Log Analytics queries, or an Azure Workbook or portal dashboard built on them.

- **One Service Level Objective** for the product (e.g. “99% of `GET /items` requests under 500 ms over 7 days”), with the query that measures it.
- **Partial failure:** with one service down (or an external API failing), the other keeps working and the affected flow fails with a timeout and a clear message, not a stack trace or a hung request.

7. Documentation

In the repository:

- **`README.md`:** what the product does, the live URL, the architecture diagram (matching the deployment), how to run it locally, run the tests and reproduce the Azure setup, and the team with each member’s GitHub username.
- **`docs/decisions.md`:** 4 to 6 short decision records (context, decision, alternatives, consequences): architecture, storage, deployment and pipeline, monitoring, and anything else that shaped the product.
- **`docs/data-model.md`:** the entities and their relationships (diagram), matching the migrations.
- **`docs/scrum/`:** Definition of Done and one file per sprint (§2). The GitHub Project stays public to the team and the professor.
- **`AI_USAGE.md`:** as in the individual assignments, one row per significant AI-assisted piece of work, including workflows, Dockerfiles and IaC.

The graded version is the commit tagged **`submission`** on `main`, pushed before the deadline (same procedure as Assignment 2 §9).

8. Sprint Review (25–26 November)

10 minutes per team, then 5 minutes of questions:

1. The product, live, from its public URL: the Sprint Goal and the items completed.
2. A change merged during the review or just before it, going through the pipeline to production (a recorded run is accepted if time is short).
3. The monitoring dashboard and the SLO.
4. What the team would do in the next sprint, and the main retrospective agreement.

Every member presents a part. Questions may be addressed to **any** member about **any** part of the system; a member who cannot explain the parts they claim to have built may receive a lower individual grade than the team.

9. Individual Contribution Statement

Each member submits, **individually on Blackboard, by the deadline**, a one-page document with:

- what they personally built (with links to their pull requests);
- the roles they held in each sprint;
- a contribution percentage for each member, themselves included, adding up to 100%, with one line of justification each.

Statements are confidential. Large and consistent differences between a member's share and the team average, together with the Git history, may raise or lower that member's grade by up to 30% of the team grade.

Deliverables

- Team repository (on GitHub, shared with the professor), commit tagged `submission`, its URL and SHA submitted on Blackboard by one member
- Live Azure URL in the README
- The documentation in §7
- Sprint Review presentation and demo (§8)
- Individual contribution statement from each member (§9)

Grading Criteria

The team grade is scored with the rubric in `GRADING_RUBRIC.md`, summarized below, then adjusted per member (§8, §9).

Category	Weight
Product, Architecture & Code Quality	25%
Cloud Infrastructure	15%
DevOps Pipeline	20%
Testing	10%
Monitoring & Reliability	10%
Scrum Process	10%
Documentation & Sprint Review	10%
Total	**100%**

Leaked credentials: as in Assignment 2, a credential in the repository or its history that was never revoked, rotated or reported caps the team grade at **40/100**.